

ISIMA 3^{ème} année – MODL/C++

TP 2 (1 séance) : Généricité

Pré-requis

On reprend le premier exercice du TP précédent. On suppose que les classes `Complexe`, `Algebrique` et `Polaire` ont été correctement définies et sont fonctionnelles, et que les moyens de conversion entre `Algebrique` et `Polaire` existent. Copiez le code source de ces classes dans le répertoire `tp_2/src` et modifiez `CMakeLists.txt` pour que CMake les prenne en compte.

L'utilisation de `Git`, au moins localement, est recommandée pour les TP de MODL/C++. Cet outil sera également disponible sur les sessions d'examen pendant l'évaluation pratique.

Exercice

- 1) Modifier la méthode `setArg` de la classe `Polaire` pour qu'elle garantisse une valeur comprise dans $[-\pi, \pi]$. Ceci peut être accompli en utilisant la fonction standard `std::remainder` (cf. <https://en.cppreference.com/cpp/numeric/math/remainder>). Test 0
- 2) Définir la classe `Ensemble`, générique sur le type de nombres Complexes à contenir. Utiliser la classe `std::vector` pour stocker les éléments. Cette classe devra fournir des itérateurs, à la manière STL (méthodes `begin()` et `end()`, type `iterator`) et une méthode `size()` qui renvoie le nombre d'éléments stockés. Tests 1–2
- 3) Implémenter pour la classe `Algebrique` la soustraction (`operator-`) et la division par un nombre réel (`operator/`). S'ils n'ont pas déjà été faits, implémenter également l'addition (`operator+`) et le produit (`operator*`). Tests 3–6
- 4) Définir une fonction générique `moyenne()` prenant en argument un ensemble de nombres et renvoyant la moyenne, supposée du même type que les nombres. Proposer une solution générique qui s'appuie sur la formule connue pour les complexes algébriques (utiliser des conversions). Penser à utiliser les opérateurs nouvellement définis. Tests 7–8
- 5) On souhaite que la fonction de moyenne s'affranchisse aussi du type du conteneur de nombres. Pour cela, la classe `Ensemble` propose une interface similaire aux conteneurs STL, à savoir des méthodes `begin()` et `end()`, ainsi que les types internes `iterator` et `value_type`. Proposer une fonction `moyenne_generique()` qui puisse fonctionner indifféremment sur les principaux conteneurs STL et la classe `Ensemble`. Tests 9–11
- 6) Implémenter une fonction `rotate()` prenant en argument un ensemble de `Polaire` et un angle `theta` (en radians) qui applique une rotation à chacun des éléments de l'ensemble. Test 12
- 7) Surcharger la fonction `rotate()` pour le type `Ensemble<Algebrique>`. Il est possible d'effectuer la rotation par une simple multiplication par un nombre tel que :

$$\begin{aligned} \text{re} &= \cos(\theta) \\ \text{im} &= \sin(\theta). \end{aligned}$$

Test 13

- 8) Définir une classe générique `RotatePivot` munie d'une fonction statique `apply` acceptant trois arguments : un ensemble de nombres, un angle `theta` en radians et un point de pivot représenté par un nombre complexe. Cette fonction applique une rotation centrale selon le pivot de l'angle `theta` à chacun des éléments de l'ensemble.

La classe doit être générique à la fois sur le type de nombres présents dans l'ensemble et, indépendamment, sur le type du pivot (elle a donc deux paramètres *template*).

Pour l'implémentation générique, appliquer la rotation en passant par le type `Polaire`. Test 14

- 9) Cette implémentation n'est pas particulièrement efficace pour les nombres complexes algébriques. Spécialiser la classe générique `RotatePivot` pour le cas d'un `Ensemble<Algebrique>`, le deuxième paramètre *template* restant indéterminé. Tests 15–16